HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND ENGINEERING



REPORT LAB 01

DEPLOY YOUR FIRST STATIC WEBSITE TO THE CLOUD

Student full name:	Tô Phương Đạt
Student ID:	24110086
Course:	Introduction to Cloud Computing
Deployment platform:	Option B — GitHub Pages
GitHub repository link:	https://github.com/DATOXIC/cloud-lab1
Public website link (HTTPS):	https://datoxic.github.io/cloud-lab1/
Submission file name:	Lab01_24110086_ToPhuongDat.zip

DEPLOYMENT EVIDENCE

E1 - Desktop Website

The full website on desktop, loaded from the public HTTPS URL, showing my name and student ID.

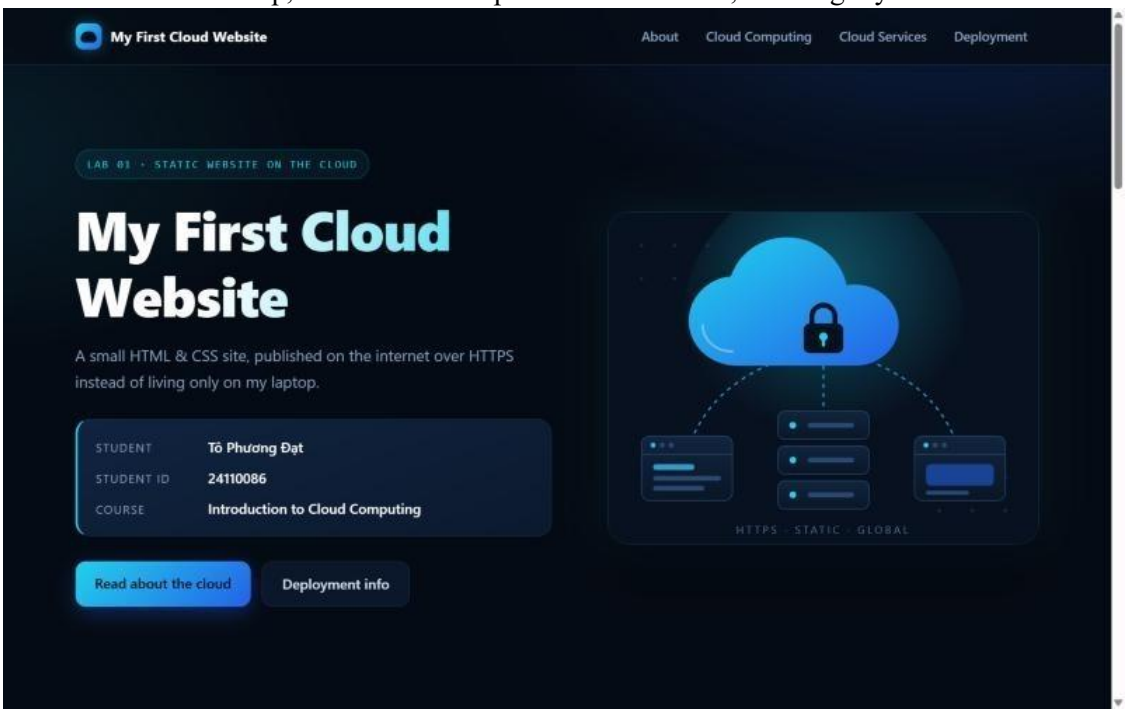


Figure 1. Desktop view – header, title and student information.

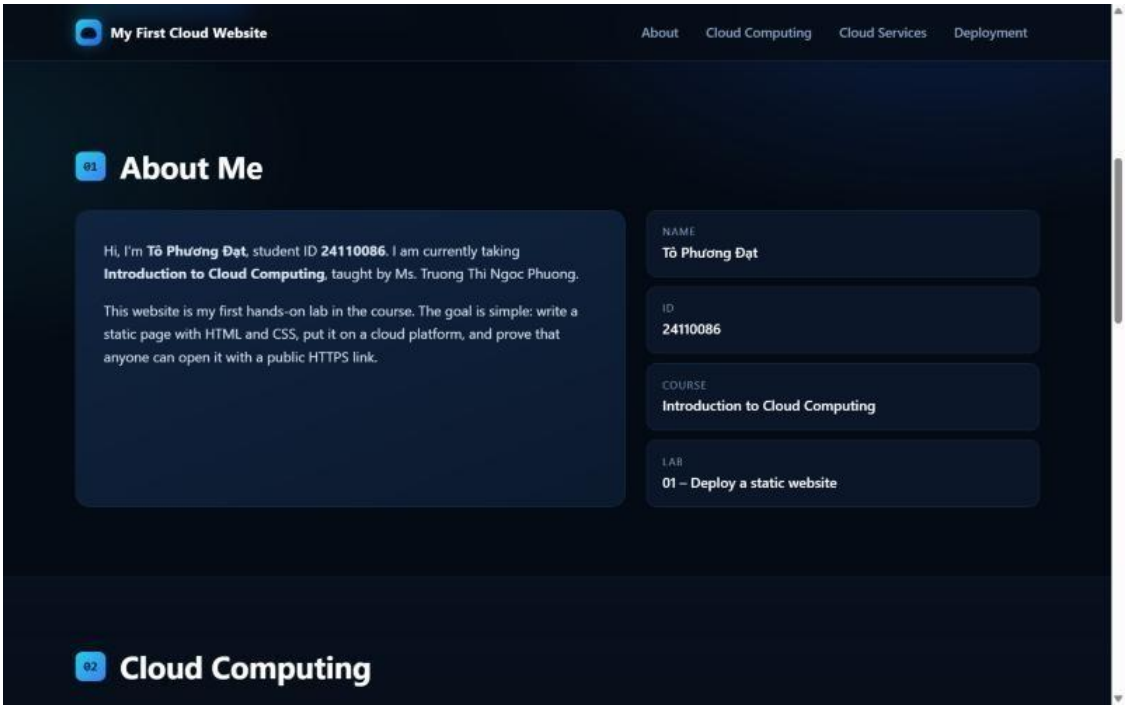


Figure 2. Desktop view – About Me section.

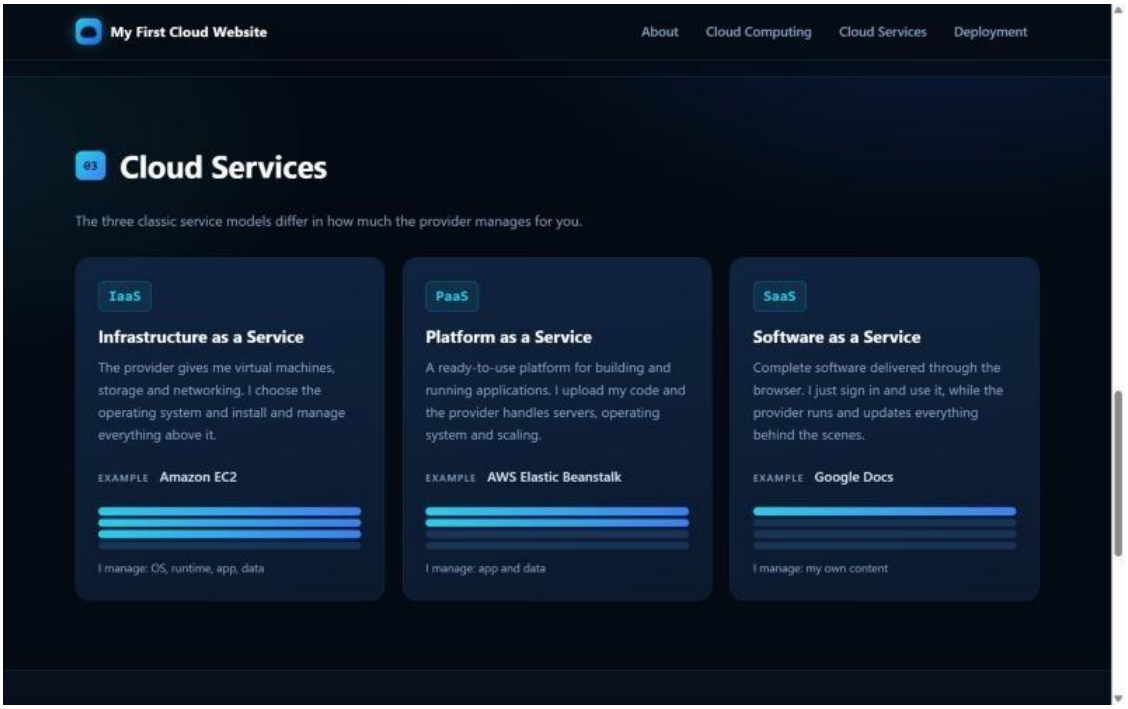


Figure 3. Desktop view – Cloud Services (IaaS, PaaS, SaaS).

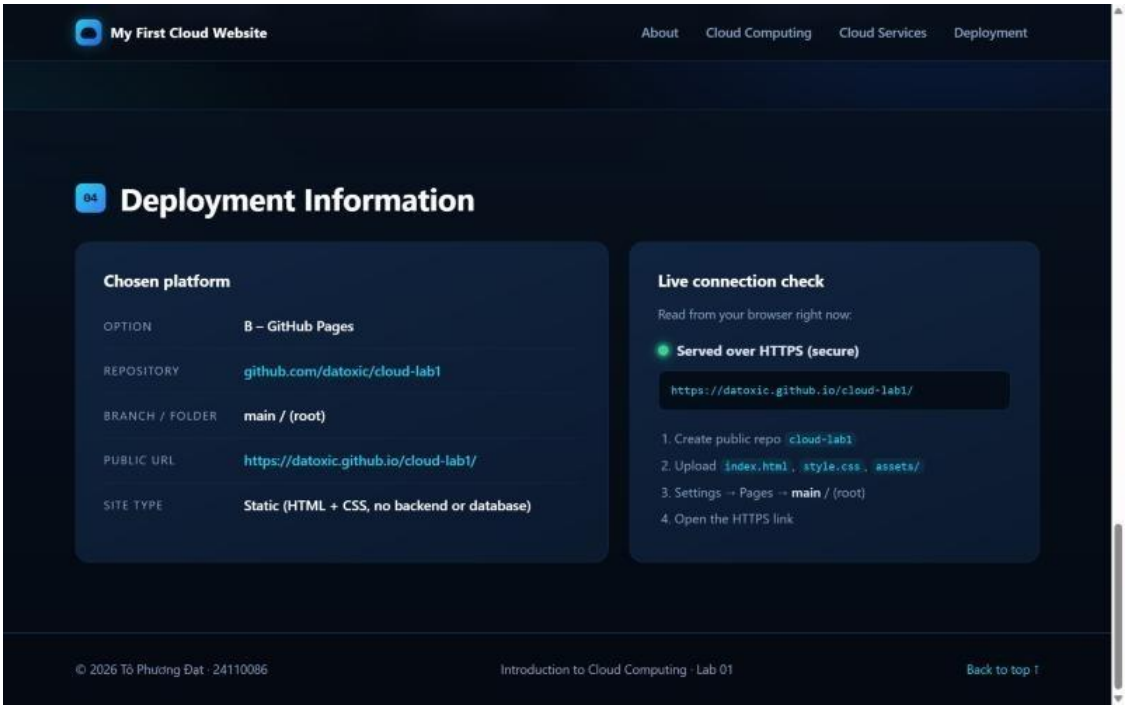
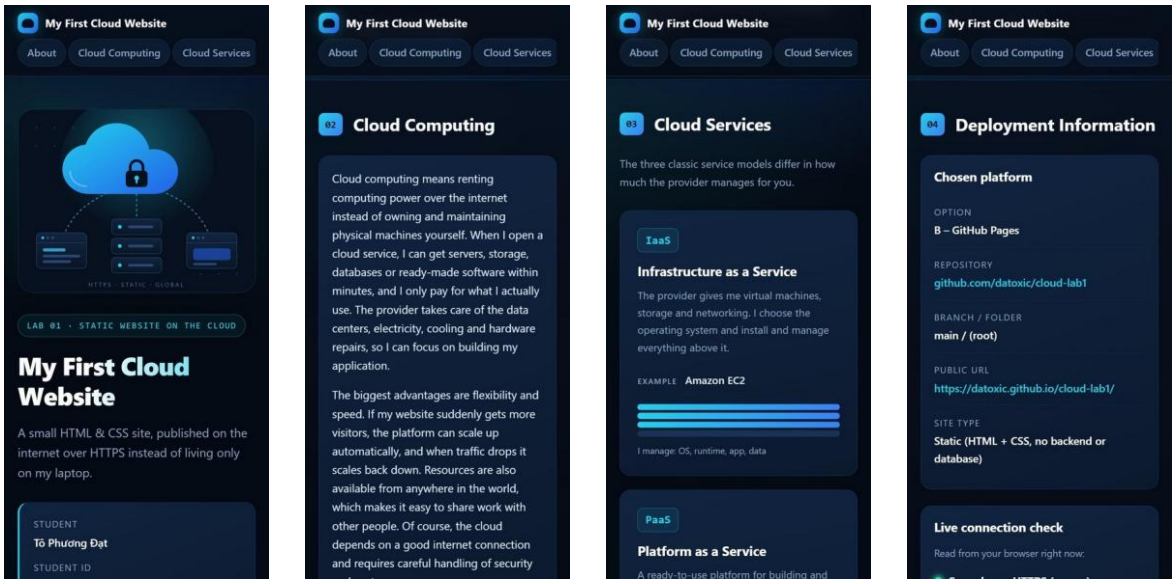


Figure 4. Desktop view – Deployment Information and footer.

E2 - Mobile Website

Responsive view in a narrow viewport (390 × 844). The layout switches to a single column and the menu becomes a horizontally scrollable row.



Figures 5–8. Mobile view: header and title, Cloud Computing, Cloud Services, Deployment Information.

E3 - Hosting Dashboard

Repository cloud-lab1 (public) and the GitHub Pages settings: source is “Deploy from a branch”, branch main, folder / (root).

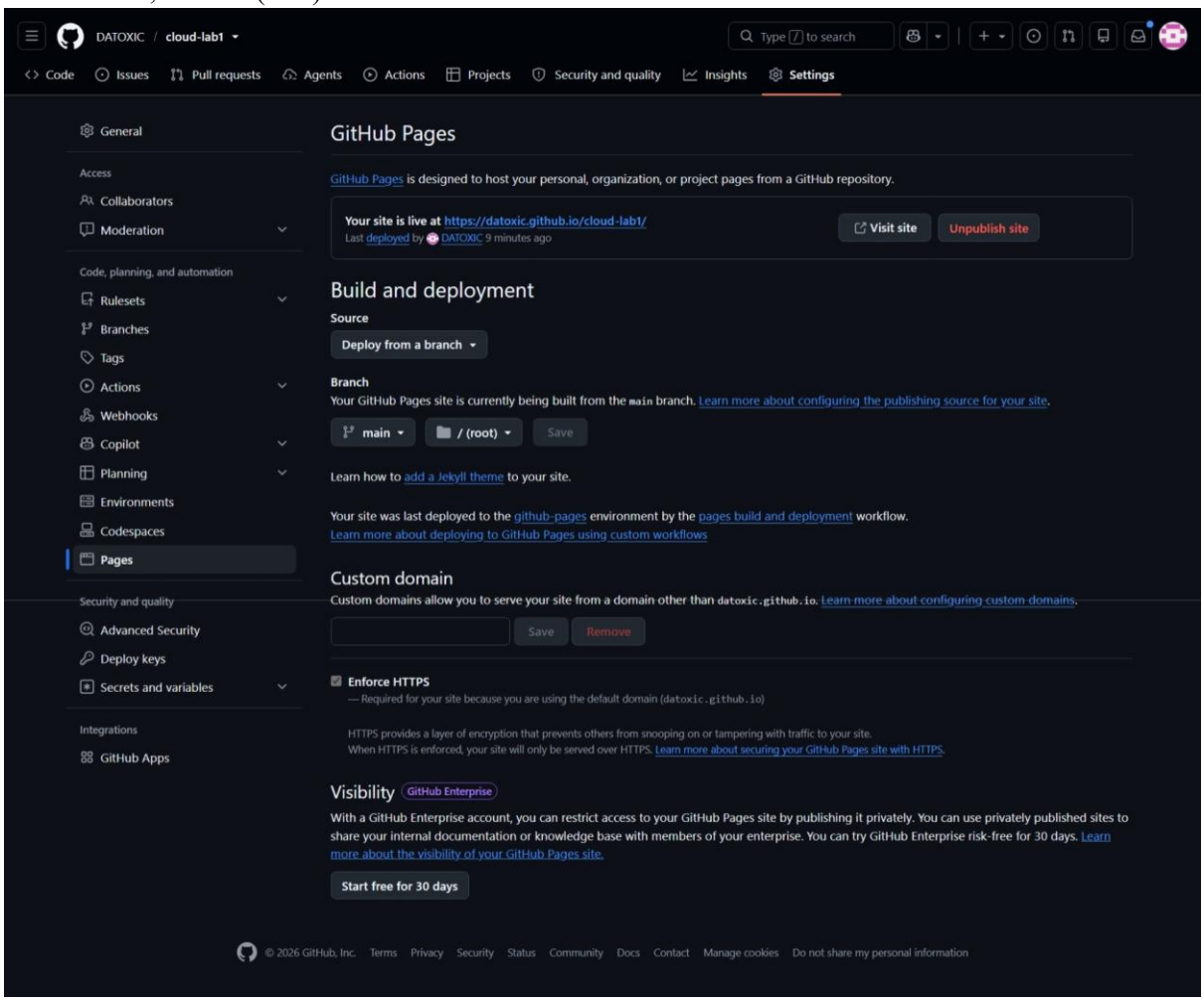


Figure 9. Settings → Pages of the repository cloud-lab1.

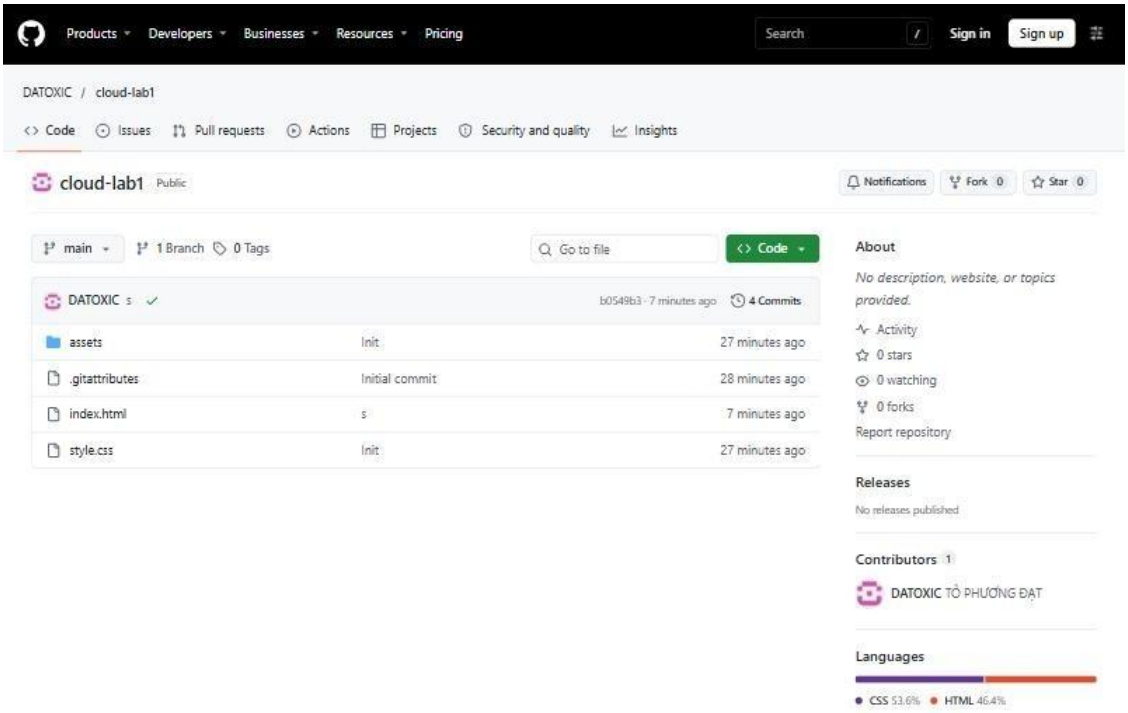


Figure 10. Public repository cloud-lab1 on GitHub (index.html, style.css, assets/).

E4 - HTTPS Verification

The browser address bar of an incognito window shows the public HTTPS URL https://datoxic.github.io/cloud-lab1/ and the page opens without signing in.

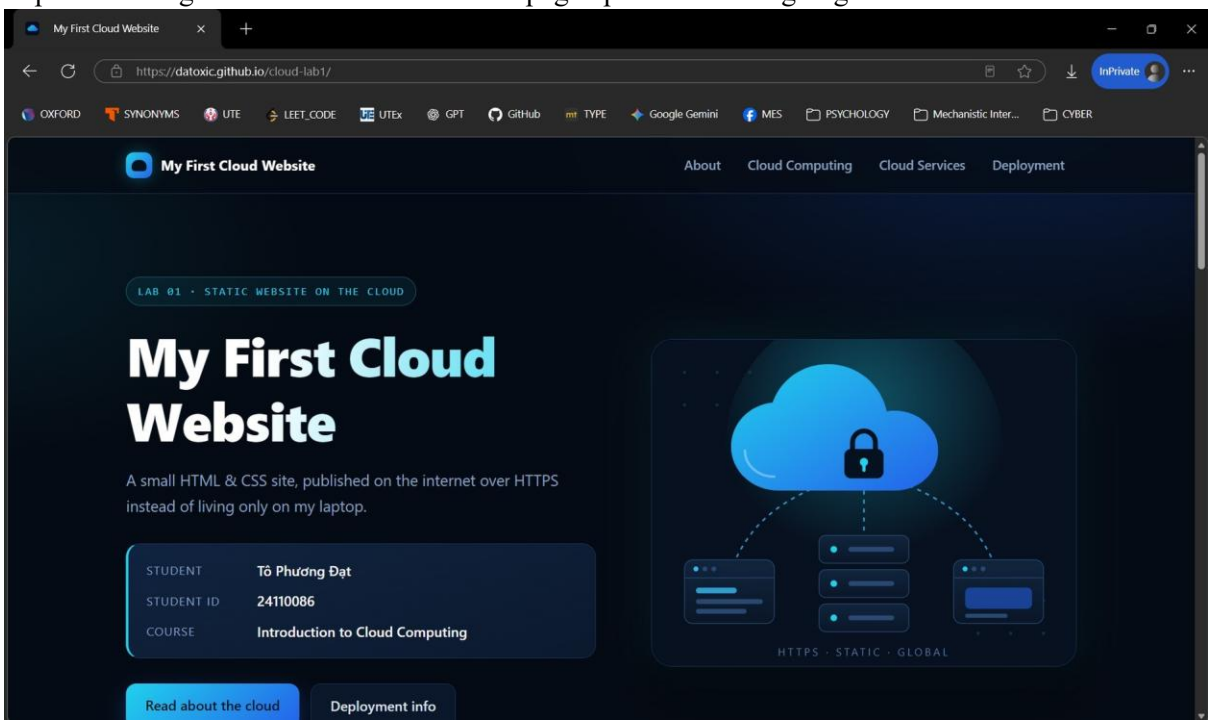


Figure 11. Address bar showing the public https:// URL.

WRITTEN ANSWERS (Q1 - Q5)

Q1. Which platform did you select, and why?

I chose GitHub Pages (Option B). I already use GitHub, and the platform is free, so I did not need a payment card or any special AWS access. The workflow is short: I created a public repository called cloud-lab1, uploaded index.html, style.css and the assets folder to the main branch, and switched on Pages from the repository settings. GitHub then built the site and gave me a ready-to-use HTTPS address. Because my files stay in a Git repository, I can also fix mistakes and republish just by committing a new version.

Q2. How does opening index.html locally differ from visiting the deployed HTTPS URL?

Opening index.html locally makes the browser read the file straight from my own disk through a file:// address. Nobody else can reach it, and it only exists on my computer. Visiting the deployed URL means the browser requests the page from a GitHub server over the internet. Anyone with the link can load it on any device, even when my laptop is turned off, and the address is a real domain name. The connection is also encrypted. To change the online version, I must commit new files instead of just saving them.

Q3. What is HTTPS, and why is it important?

HTTPS is the normal web protocol, HTTP, running inside a TLS encrypted channel. It matters for three reasons. Encryption stops people on the same network from reading what is exchanged. Integrity means the page cannot be silently altered while it travels to the browser. Authentication means a certificate shows that the site really belongs to the domain I typed. Browsers warn users about sites without HTTPS and some features refuse to work without it, so it is a basic requirement for any public website.

Q4. Does the selected hosting platform fit IaaS, PaaS or SaaS directly? Explain its characteristics.

GitHub Pages does not match one model perfectly, but it is closest to PaaS. I never create a virtual machine, install an operating system or configure a web server; I only provide my website files and the platform runs them. It is not IaaS because I have no control over the underlying machines or network. It is not classic SaaS either, because I am not just using a finished application, I am publishing my own content. Its main traits are static hosting only, automatic HTTPS, and deployment triggered from a Git branch.

Q5. What changes would be needed for authentication or a database?

GitHub Pages can only deliver static files, so a login system or database needs a backend somewhere else. I could write a server application, for example with Node.js, or rely on a managed service such as AWS Amplify, Firebase or Cloudflare Workers. Authentication would require a service that checks credentials, stores passwords safely and issues sessions or tokens. A database like PostgreSQL or DynamoDB would hold user data behind an API. My static pages would then call that API with JavaScript, and I would also need access control and secret management.